M T W T F S S
Page No.
Date
YOUVA

Assignment - 1

- Aim :
  To write a C program to perform basic matrix operations such as addition, subtraction, multiplication, and transpose of a matrix using two dimensional arrays.

- objectives
  - To understand the concept of two dimensional arrays in C.
  - To perform arithemetic operations on matrices.
  - To implement matrix addition and subtraction.
  - To implement matrix multiplication using nested loops
  - To find the transpose of a matrix.
  - To develop programming skills using arrays & loops

- Theory
  A matrix is rectangular arrangement of elements organized in rows & columns. In C programming, matrices are represented using two dimensional arrays. Matrix operations are widely used in computer graphics, image processing, machine learning, scientific computing and engineering applications.

1. Matrix Addition
   Condition: Both matrices must have the same number of rows & columns.
   Formula: $C[i][j] = A[i][j] + B[i][j]$
2. Matrix Subtraction
   Condition: Both matrices must have the same dimensions.
   Formula: $C[i][j] = A[i][j] + B[i][j]$

3. Matrix Multiplication

Condition: Number of Columns of the ~~first~~ matrix must be equal to the number of rows of the second matrix.

Formula: $C[i][j] = \sum A[i][k] \times B[k][j])$ where $k=0$ to $n-1$

4. Transpose of Matrix

The transpose of matrix is obtained by interchanging its rows & columns.

Formula: $AT[i][j] = A[j][i]$

o Frequency Asked Questions

1. Explain matrix addition, subtraction, multiplication, and transpose with suitable examples and state the conditions required for each operation.

->

1. Matrix Addition

Matrix addition is the process of adding the corresponding elements of two matrices. The sum of two matrices is obtained by adding each element in one matrix to the corresponding element in other matrix.

Condition:

Two matrices can be added only if they have same order (same number of rows & columns).

Ex:

$$A\begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix}, \quad B=\begin{bmatrix} 5 & 1 \\ 6 & 2 \end{bmatrix}$$

Then,

$$A+B = \begin{bmatrix} 2+5 & 4+1 \\ 1+6 & 3+2 \end{bmatrix} = \begin{bmatrix} 7 & 5 \\ 7 & 5 \end{bmatrix}$$

2. Matrix Subtraction

Matrix subtraction is process of subtracting the corresponding elements of one matrix from another.

Condition:

Both matrices must have the same order.

Ex:

$$A = \begin{bmatrix} 8 & 6 \\ 5 & 9 \end{bmatrix}, \quad B = \begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}$$

Then,

$$A - B = \begin{bmatrix} 8-3 & 6-2 \\ 5-1 & 9-4 \end{bmatrix} = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}$$

3. Matrix Multiplication

Matrix multiplication is performed by multiplying the rows of first matrix with the columns of the second matrix and adding the products.

Condition:

If matrix A is of order $m \times n$ and matrix B is of order $n \times p$, then multiplication AB is possible. The resulting matrix will be of order $m \times p$.

Ex:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 1 \\ 0 & 5 \end{bmatrix}$$

Then

$$AB = \begin{bmatrix} (1\times2+2\times0) & (1\times1+2\times5) \\ (3\times2+4\times0) & (3\times1+4\times5) \end{bmatrix} = \begin{bmatrix} 2 & 11 \\ 6 & 23 \end{bmatrix}$$

4. Transpose of Matrix

The transpose of matrix is obtained

by interchanging its rows and columns. If A is a matrix, its transpose is denoted by A^T.

Condition:

The transpose can be found for any matrix, regardless of its order.

Ex:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

2. Describe how two-dimensional arrays are used to represent matrices in C programming. Explain the process of accessing and manipulating matrix elements nested loops.

=>

A matrix is a rectangular arrangement of numbers in rows and columns. In C programming, a matrix is represented using a two-dimensional (2D) array because it stores data in rows and columns, just like a mathematical matrix.

Representation of matrix using a 2D Array.

Declaration:

datatype array_name [rows] [columns];

Ex: int matrix [3] [3]

Here,

3 represents number rows

8 Columns.
Each element is accessed using two indices.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Accessing matrix elements using nested loops.
Since matrix has rows & columns, nested loops are used to access each elements
- The outer loop controls the rows.
- The inner loop controls the columns.

Ex:
```
#include <stdio.h>
int main() {
    int matrix[2][2] = {{1,2},{3,4}};
    int i, j;
    for (i = 0; i < 2; i++) {
        for(j = 0; j < 2; j++) {
            printf("%d", matrix[i][j]);
        }
    }
    return 0;
}
```

Manipulating matrix elements Using Nested loops.
- Reading matrix elements from the user.
- Displaying the matrix.
- Adding or subtracting matrices.
- Multiplying matrices.
- Finding the transpose of matrix.
- Updating or modifying individual elements.

3. Write the algorithm for performing matrix multiplication and explain why compatibility between matrices is necessary before multiplication.

→

Algorithm for matrix Multiplication

① Start
② Read numbers of rows & columns of A.
③ Read number of rows & Columns of B.
④ Check whether the number of columns of the first matrix equals the number of rows of the second matrix.
- If condition is not satisfied display "matrix multiplication is not possible" Stop.
⑤ Continue, Read elements of matrices A & B.
⑥ Initialize all element of the resultant matrix C to zero.
⑦ Multiply the matrices using three nested loops.
⑧ Compute the sum of the products of corresponding elements:
⑨ Display the resultant matrix C.
⑩ Stop.

Compatibility of Matrix multiplication is possible only when the number of columns in the first matrix is equal to the number of rows in the second matrix.
This is called the compatibility condition for matrix multiplication.

Suppose matrix A is of order m x n and matrix B is of order p x q. Multiplication AB is possible only if:
n = p
This condition is necessary because each element of the resultant matrix is obtained by multiplying the elements of rows of the first matrix with the corresponding elements of a column of the second matrix.
When the matrices are compatible, the resulting matrix has the order.
m x q
Ex: Matrix A: 2 x 3
Matrix B: 3 x 4
Since number of columns in A(3) is equal to number of rows in B(3), multiplication is possible.
If matrix A is 2 x 3 and matrix B is 2 x 4 multiplication is not possible because 3 ≠ 2.

4. Discuss the application of matrix operations in real-world domains such as computer graphics, image processing, machine learning, engineering and scientific computing.

→

Matrix operations are widely used in science, engineering, and computer technology because they provide an efficient way to organize, process, and analyze large amounts of data.

① Computer Graphics

In computer graphics, matrices are used to perform transformation on objects such as translation, rotation, scaling, and reflection. Every object on computer screen or in a 3D game is represented using matrices. Matrix multiplication helps change the position, size, and orientation of objects quickly and accurately.

② Image Processing

Digital images are stored as matrices of pixels, where each element represents the intensity or color of a pixel. Matrix operation are used to process images by applying filters, adjusting brightness and contrast, sharpening images, reducing noise, detecting edges, and compressing images. These techniques are widely used in medical imaging, satellite imaging, photography, and facial recognition systems.

③ Machine learning

ML algorithms represents data, features, and model parameters as matrices. Matrix multiplication is used during the training of models, especially in neural networks, to calculate outputs and update weights efficiently. Matrix operations helps process large datasets quickly, making them essential for application such as speech

recognition.

④ Engineering

Engineers use matrices to solve systems of linear equations that arise in structural analysis, electrical circuit design, mechanical systems, and control systems. Matrix methods simplify complex engineering calculations and improve computational efficiency.

⑤ Scientific Computing

In scientific computing, matrices are used to model and solve mathematical problems in physics, chemistry, biology and environmental science. Matrix operations help simulate physical systems, analyze experimental data, solve differential equation, perform weather forecasting, and carry out numerical analysis.

5. Explain the role of nested loops in implementing matrix operations in C. Illustrate how nested loops are used for addition, multiplication & transpose of matrices.

→

A matrix is a collection of elements arranged in rows and columns. In C programming, matrices are represented using two-dimensional arrays. Since a matrix has both rows and columns, nested loops are used to access and manipulate its element efficiently.

The outer loop controls the rows, while the inner loop controls the rows, while the inner loop controls the columns. By using nested loops, every elements of the matrix can be processed one by one. Nested loops are essential for performing matrix operations such as addition, multiplication, and transpose. because these operations require systematic access to all matrix elements

① Matrix Addition :

In matrix, the corresponding element of two matrices are added together. Two nested loops are used to visit each row and column, and sum is stored in third matrix.

② Matrix multiplication : requires three nested loops because each element of resulting matrix is calculated by multiplying the elements of a rows of the first matrix with the corresponding elements of column of the second matrix and then adding the products.

③ Matrix Transpose

The transpose of a matrix is obtained by interchanging its rows & columns. Two nested loops are used to copy each element from its original position to the transposed position.

16/4/24

Code:

```
#include <stdio.h>
void accept(int a[10][10], int m, int n)
{
   int i, j;
   printf("\nEnter matrix elements:\n");
   for (i = 0; i < m; i++)
   {
      for (j = 0; j < n; j++)
      {
         scanf("%d", &a[i][j]);
      }
   }
}

void display(int a[10][10], int m, int n)
{
   int i, j;
   printf("\nMatrix:\n");
   for (i = 0; i < m; i++)
   {
      for (j = 0; j < n; j++)
      {
         printf("%4d", a[i][j]);
      }
      printf("\n");
   }
}

void addition(int a[10][10], int b[10][10], int m, int n)
{
   int c[10][10], i, j;

   for (i = 0; i < m; i++)
   {
      for (j = 0; j < n; j++)
      {
         c[i][j] = a[i][j] + b[i][j];
      }
   }

   printf("\nAddition of matrices:\n");
   display(c, m, n);
}

void subtraction(int a[10][10], int b[10][10], int m, int n)
{
```

```
    int c[10][10], i, j;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            c[i][j] = a[i][j] - b[i][j];
        }
    }

    printf("\nSubtraction of matrices:\n");
    display(c, m, n);
}

void multiplication(int a[10][10], int b[10][10], int m, int n)
{
    int c[10][10], i, j, k;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            c[i][j] = 0;
            for (k = 0; k < n; k++)
            {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }

    printf("\nMultiplication of matrices:\n");
    display(c, m, n);
}

void transpose(int a[10][10], int m, int n)
{
    int t[10][10], i, j;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            t[j][i] = a[i][j];
        }
    }

    printf("\nTranspose of matrix:\n");
```

```
    display(t, n, m);
}

int main()
{
    int a[10][10], b[10][10];
    int m, n, ch;

    printf("Enter number of rows and columns: ");
    scanf("%d%d", &m, &n);

    printf("\nEnter First Matrix:\n");
    accept(a, m, n);

    printf("\nEnter Second Matrix:\n");
    accept(b, m, n);

    do
    {
        printf("\n------ MENU ------");
        printf("\n1. Addition");
        printf("\n2. Subtraction");
        printf("\n3. Multiplication");
        printf("\n4. Transpose of First Matrix");
        printf("\n5. Display Matrices");
        printf("\n6. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &ch);

        switch (ch)
        {
        case 1:
            addition(a, b, m, n);
            break;

        case 2:
            subtraction(a, b, m, n);
            brea

        case 3:
            if (m == n)
                multiplication(a, b, m, n);
            else
                printf("\nMultiplication is supported here only for square matrices.\n");
            break;

        case 4:
```

```
            transpose(a, m, n);
            break;

        case 5:
            printf("\nFirst Matrix:");
            display(a, m, n);

            printf("\nSecond Matrix:");
            display(b, m, n);
            break;

        case 6:
            printf("\nProgram terminated.");
            break;

        default:
            printf("\nInvalid Choice!");
        }

    } while (ch != 6);

    return 0;
}
```

Input and Output:

```
Enter number of rows and columns:2 2


Enter First Matrix:

Enter matrix elements:
1 2 3 4


Enter Second Matrix:

Enter matrix elements:
5 6 7 8


------ MENU ------
1. Addition
2. Subtraction
3. Multiplication
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:1
```

```
Addition of matrices:

Matrix:
   6   8
  10  12

------ MENU ------
1. Addition
2. Subtraction
3. Multiplication
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:2


Subtraction of matrices:

Matrix:
  -4  -4
  -4  -4

------ MENU ------
1. Addition
2. Subtraction
3. Multiplication
```

```
3. Multiplication
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:3


Multiplication of matrices:

Matrix:
  19  22
  43  50

------ MENU ------
1. Addition
2. Subtraction
3. Multiplication
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:4


Transpose of matrix:

Matrix:
   1   3
   2   4
```

```
Enter your choice:5


First Matrix:
Matrix:
   1   2
   3   4

Second Matrix:
Matrix:
   5   6
   7   8

------ MENU ------
1. Addition
2. Subtraction
3. Multiplication
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:6


Program terminated.
Process finished with exit code 0
```

```
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:5


First Matrix:
Matrix:
   1   2
   3   4

Second Matrix:
Matrix:
   5   6
   7   8

------ MENU ------
1. Addition
2. Subtraction
3. Multiplication
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:6


Program terminated.
Process finished with exit code 0
```